CPSC 250: Programming for Data Manipulation

Course Roadmap and Zybooks Structure

Christopher Newport University

Summer 2026

1 Course Philosophy

CPSC 250 bridges introductory Python programming and modern computational problem solving. The course begins by reinforcing material from CPSC 150/150L, then moves into object-oriented programming, algorithms, recursion, scientific computing, data analysis, and visualization.

The goal is not simply to learn more Python syntax. The goal is to develop lasting computational problem-solving skills.

2 How the CPSC 250 Zybook Is Organized

The CPSC 250 Zybook is custom organized from material students may recognize from CPSC 150, but the ordering and emphasis have been changed to support the goals of this course.

Some early material is review and reinforcement. Later chapters introduce major new topics.

Review and Reinforcement from CPSC 150

CPSC 150 Zybook	Topic	CPSC 250 Placement
Ch. 1	Introduction	Ch. 1
Ch. 3	Variables / Expressions	Ch. 2
Ch. 4	Primitive Types	Ch. 3
Ch. 6	Branching	Ch. 4
Ch. 7	Loops	Ch. 5
Ch. 5 and Ch. 8	Functions	Ch. 6
Ch. 9	Strings	Ch. 7
Ch. 10	Lists / Dictionaries	Ch. 8
Ch. 12	Modules	Ch. 11
Ch. 11	Files	Ch. 12

This material is not intended to completely reteach CPSC 150. Instead, it reinforces the programming tools students are expected to carry forward.

Major New CPSC 250 Topics

CPSC 250 Chapter	Topic
Ch. 9	Classes and Objects
Ch. 13	Inheritance and Polymorphism
Ch. 14	Recursion
Ch. 15	Searching and Sorting Algorithms
Ch. 30	Scientific Computing and Data Analysis

3 Course Structure

Section I — Review and Reinforcement

- variables, expressions, and primitive data types
- branching and loops
- functions
- strings, lists, and dictionaries
- files and modules

Section II — Object-Oriented Programming

- classes and objects
- constructors
- instance variables
- methods
- encapsulation
- operator overloading

Section III — Inheritance and Polymorphism

- inheritance
- derived classes
- method overriding
- polymorphism

Section IV — Algorithms and Problem Solving

- recursion
- searching algorithms
- sorting algorithms
- problem decomposition
- computational efficiency

Section V — Scientific Computing and Data Analysis

- NumPy
- pandas
- matplotlib
- data visualization
- curve fitting
- data analysis

4 Lecture and Laboratory Relationship

CPSC 250 and CPSC 250L are designed to complement one another.

CPSC 250 focuses on concepts, computational thinking, algorithms, problem solving, and scientific/data-oriented programming.

CPSC 250L emphasizes active in-class programming, debugging, GitHub workflows, feature branches, version control, command-line literacy, and professional software development practices.

5 Development Environment

Students will use:

- Python 3
- PyCharm Community Edition
- GitHub
- Git

Students enrolled in either CPSC 250 or CPSC 250L should complete the development environment setup process before the end of the first week.

6 Summer Course Rhythm

The summer course is highly compressed and moves quickly.

Morning	CPSC 250 lecture
Afternoon	CPSC 250L laboratory
Fridays	Tests / assessments
June 19	No classes, Juneteenth

7 Grading and Course Policies

Detailed grading policies, assignment policies, attendance expectations, and course requirements are described in the official course syllabi.

The syllabi are the authoritative source for grading weights, assignment policies, late work policies, attendance expectations, academic integrity policies, and laboratory requirements.

8 Final Thoughts

Programming is learned by actively writing, testing, debugging, and refining code.

Students are encouraged to experiment with code examples, ask questions frequently, practice regularly, become comfortable debugging, and develop confidence with professional software tools.